UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

(Developing reliable code)

Class design by contract

Supplementary readings: (

: chapt. 3), (

: chapter 11)

1

Computing with rationals

- Develop a program able to perform basic arithmetic operations and comparisons with rational numbers.

Examples of the required operations are:

i) 1/4 + 1/4 -> 1/2
ii) -(1/2) * 1/2 -> - 1/4
iii) 1/2 > 1/3 -> true
iv) 2/4 = 1/2 -> true

2

Development of the class Fraction representing rationals

- Representation:
  n – numerator
  d – denominator
  d>0
  gcd (|n|, d) = 1 (máximo divisor comum – gcd in English) Ex. 2/4 == 1/2

Class invariant

(condição invariante de instância)

- Under the above invariant, operations can be given by:
  n1/d1 + n2/d2 = (n1d2 + n2d1) / (d1 * d2);
  n1/d1 – n2/d2 = (n1d2 - n2d1) / (d1 * d2);
  n1/d1 * n2/d2 = n1 * n2 / (d1 * d2);
  (n1/d1) / (n2/d2) = n1* d2 / (n2* d1); with n2 ≠ 0
  (n1/d1) == (n2/d2) <=> n1 == n2 and d1 == d2
  n1/d1 < n2/d2 <=> n1d2 < n2d1

3

Class invariant

(condição invariante de instância)

```
/** @version 1.0 */
public class Fraction {
   private int num, den;
   public Fraction() { this(0); }
   public Fraction(int n) { num= n; den = 1; }
   public Fraction (int n, int d) {
      if (d == 0) {
          System.err.print("Fraction.denominator() cannot be zero");
          System.exit(0);      // We will have to improve this later
      }
      if (d < 0) {
         this.num = -n;
         this.den = -d;
      }  else {
       num = n;          // same as this.num =n;
       den = d;
      }
      int k =gcd(Math.abs(num), den);
      num /= k;
      den /= k;
   }
```

d > 0

gcd (|n| , d) = 1

eg.: 8/16 == 1/2, k=8

4

```
/** retorna uma nova instância da classe com o resultado da
    multiplicação do recetor this pelo argumento that
        @param that uma Fraction
        @return  n1/d1 * n2/d2 = n1n2 / (d1 * d2);
        @version 1.0
    */
 public Fraction mux(Fraction that) {
    int n = this.num * that.num;
    // ou int n = num * that.num;
    int d = this.den * that.den;
    return new Fraction(n, d);
}
```

By creating a new object, the class invariant is enforced

```
class Main {
        void main() {
            //..
            Fraction s = f.mux(g);
        }
}
```

1/2 * 1/3 = 1/6

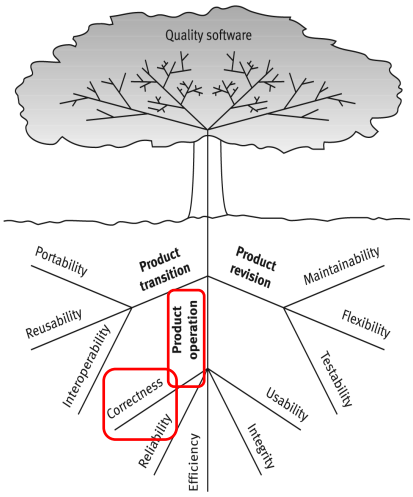
5

```
/**    Calcula o máximo divisor comum dos seus parâmetros seguindo
       Euclides
          @param m        inteiro positivo
          @param n        inteiro positivo
          @return  inteiro com o máximo divisor comum entre m e n
          @pre     m >0 && n >0
          @post    return == gcd(m, n)

 */
   private int gcd(int  m, int  n){
    int r;
    while (( r = m % n ) != 0 ) {
          m = n;
          n = r;
    }
    return n;
   }
} // end of class Fraction
```

6

McCall Quality Factors



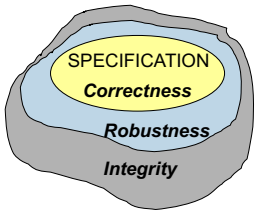
7

McCall software quality factor model

Quality Categories	Quality Factors	Broad Objectives
Product operation	Correctness	Does it do what the customer wants?
	Reliability	Does it do it accurately all of the time?
	Efficiency	Does it quickly solve the intended problem?
	Integrity	Is it secure?
	Usability	Can I run it?
Product revision	Maintainability	Can it be fixed?
	Testability	Can it be tested?
	Flexibility	Can it be changed?
Product transition	Portability	Can it be used on another machine?
	Reusability	Can parts of it be reused?
	Interoperability	Can it interface with another system?

8

Specifications in the basis of software quality



- Correctness
  The ability of a software to perform according to specification, in the cases defined by the specification
- Robustness
  The ability of a software to react in a reasonable manner to cases not covered by the specification.
- Integrity
  The ability of a software to react in a reasonable manner to cases of unauthorized access and modification.

9

Correctness

- An implementation is *correct* relative to a specification
- Total correctness:
  Any execution of instructions *A* started in a state satisfying *P* will terminate in a state satisfying *Q*, where

  P, *Q* are assertions

10

Assertions

- Formally, assertions are first-order logic formulae
- In a programming language, assertions are Boolean expressions that can contain variables, arithmetic and boolean statements and, possibly, user-defined functions

11

The Hoare triple

$$\{P\}\ A\ \{Q\}$$

- Total correctness:
  Any execution of A started in a state satisfying P will terminate in a state satisfying Q

  Ex: $\{n > 5\}\ n := n + 9\ \{n > 13\}$

12

Specs vs Implementations

- Specs:

  pre: `x==0 /\ y==3`

  post: `x==3 /\ y==0`

- Possible implementations

```
i) x=3;  y=0
ii) x=x+y; y=x*y
iii) t=x; x=y; y=t;
```

13

Design by contract

- A design principle in which the invocation of a method is viewed as a contract between a client and the server (or supplier), with each having explicitly stated responsibilities

- A client of method *m* is any method that uses the facilities of *m*

14

The contract

- If the preconditions are satisfied, then the server guarantees that the post conditions will be satisfied when the method completes
- If the preconditions are not satisfied, that is, if the client does not meet his end of the contract, then the server promises nothing

15

The contract

	OBLIGATIONS	BENEFITS
Client	PRECONDITION	POSTCONDITION
Supplier	POSTCONDITION	PRECONDITION

16

```
/**    Calcula o máximo divisor comum dos seus parâmetros seguindo
       Euclides
           @param m          inteiro positivo
           @param n          inteiro positivo
           @pre      m >0 && n >0
           @post     return == gcd(m, n)
           @return   inteiro com o máximo divisor comum entre m e n
 */
   private int gcd(int  m, int  n){
    int r;
    while (( r = m % n ) != 0 ) {
           m = n;
           n = r;
    }
    return n;
   }
} // end of class Fraction
```

17

Properties of a contract

- Is part of the software
- Binds two parties: supplier, client
- Is explicitly written
- Specifies mutual obligations and benefits
- Usually maps obligation for one of the parties into benefit for the other, and conversely
- Has no hidden clauses: obligations are those specified

18

The objective of design by contract

- Delineate, clearly and explicitly, responsibilities between client of a method and its supplier
- Ensure that any possible run time error will be detected with minimal explicit error checking

19

The objective of programming by contract

- Test for every possible error condition only once (for program efficiency)
- Achieve a balance between program reliability and maintainability

20

The class invariant in design by contract

- *Invariants* should be enforced at all times during the live of an object
  - If default initialization of instance variables violates class invariant, an explicit constructor is needed.
  - Invariants apply to the current methods but also to any methods added later
- Correctness of a method m:

```
{P and Inv}  m_body  {Q and Inv}
```

21

So, what's a bug?

https://www.dbvis.com/thetable/why-are-they-called-bugs/

- A run-time assertion violation is a manifestation of a  in the software
  - Precondition violation: Bug in the client
  - Postcondition violation: Bug in the supplier
  - Invariant violation: Bug in the supplier

22

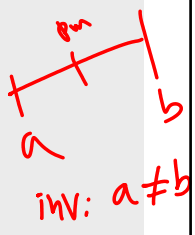
So, what's a bug?

- A run-time assertion violation is a manifestation of  in the software.
- Error: A wrong decision made during software development.
- Defect: A property of a software that may cause it to depart from its intended behavior.
- Fault: The event of a software departing from its intended behavior.

23

Check point: what's wrong, if any thing?

```
public class Main{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Ponto a = new Ponto(sc.nextDouble(), sc.nextDouble());
        Ponto b = new Ponto(sc.nextDouble(), sc.nextDouble());
        sc.close();
        if (a.getX()==b.getX()&&a.getY()==b.getY()) {
             System.out.println("inv.");
             System.exit(0);
        }
        SegmentoReta sr = new SegmentoReta(a,b);
        System.out.println(sr.pontoMedio());
    }
}
```



24

Check point: what's wrong?

```
public class Main {
    public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
        Ponto a = new Ponto(sc.nextDouble(), sc.nextDouble());
        Ponto b = new Ponto(sc.nextDouble(), sc.nextDouble());
       sc.close();
        if (a.getX()==b.getX()&&a.getY()==b.getY()) {
            System.out.println("inv.");
            System.exit(0);
        }
        // Violação que o compilador aceita:
        SegmentoReta sr = new SegmentoReta(a,a);
        System.out.println(sr.pontoMedio());
        }
    }
```

The test of *class invariants* is a responsibility of the *class*

Don't use == for doubles